

Competitive Programming

Section 26 Editorial: Tree DP Basics

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
2	A. Festival Invite	4
2.1	Restatement	4
2.2	Constraints and traps	4
2.3	Observations	4
2.4	Algorithm	4
2.5	Correctness proof	4
2.6	Complexity	4
2.7	Full C++ solution	5
2.8	Full Python solution	6
2.9	Common wrong answers	6
3	B. Road Pairing	8
3.1	Restatement	8
3.2	Observations	8
3.3	Algorithm	8
3.4	Correctness proof	8
3.5	Complexity	8
3.6	Full C++ solution	8
3.7	Full Python solution	10
4	C. Tree Colorings	11
4.1	Restatement	11
4.2	Observations	11
4.3	Algorithm	11
4.4	Correctness proof	11
4.5	Complexity	11
4.6	Full C++ solution	11
4.7	Full Python solution	12
5	Testing and carry-forward	14

1 Editorial overview

Problem	State	Combination
A. Festival Invite	choose or skip node	sum child optima
B. Road Pairing	match node to zero/one child	best child upgrade
C. Tree Colorings	color of node	product over children

All solutions root the tree at vertex 1 and process nodes in postorder.

2 A. Festival Invite

2.1 Restatement

Choose a set of vertices with maximum total happiness such that no chosen vertices are adjacent.

2.2 Constraints and traps

n is large, so brute force over subsets is impossible. A chain can be long, so recursive DFS may overflow in Python or C++ environments.

2.3 Observations

If u is invited, none of its children may be invited. If u is not invited, each child subtree can choose independently whether to invite the child.

2.4 Algorithm

```
root the tree
for u in postorder:
    take[u] = h[u] + sum(skip[v])
    skip[u] = sum(max(take[v], skip[v]))
print max(take[root], skip[root])
```

2.5 Correctness proof

Theorem. $take[u]$ and $skip[u]$ have the stated optimal meanings.

Proof. Induct on subtree height. Leaves are immediate. For an internal node u , if u is taken, every child must be skipped, and child subtrees are independent. If u is skipped, every child can use its better state. By the induction hypothesis, the child values are optimal, so the formulas are optimal. $\square$

2.6 Complexity

Time: $O(n)$
Memory: $O(n)$

2.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<long long> h(n);
    for (long long &x : h) cin >> x;
    vector<vector<int>> adj(n);
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        cin >> u >> v;
        --u; --v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> parent(n, -1), order;
    order.push_back(0);
    parent[0] = 0;
    for (int i = 0; i < (int)order.size(); i++) {
        int u = order[i];
        for (int v : adj[u]) if (parent[v] == -1) {
            parent[v] = u;
            order.push_back(v);
        }
    }
    vector<long long> take = h, skip(n, 0);
    for (int idx = n - 1; idx >= 0; idx--) {
        int u = order[idx];
        for (int v : adj[u]) if (parent[v] == u) {
            take[u] += skip[v];
            skip[u] += max(take[v], skip[v]);
        }
    }
    cout << max(take[0], skip[0]) << '\n';
    return 0;
}
```

2.8 Full Python solution

```
import sys

def postorder(adj: list[list[int]], root: int = 0) -> tuple[list[int], list[int]]:
    n = len(adj)
    parent = [-1] * n
    order = [root]
    parent[root] = root
    for u in order:
        for v in adj[u]:
            if parent[v] == -1:
                parent[v] = u
                order.append(v)
    return parent, order[::-1]

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    h = [next(it) for _ in range(n)]
    adj = [[] for _ in range(n)]
    for _ in range(n - 1):
        u = next(it) - 1
        v = next(it) - 1
        adj[u].append(v)
        adj[v].append(u)
    parent, order = postorder(adj)
    take = h[:]
    skip = [0] * n
    for u in order:
        for v in adj[u]:
            if parent[v] == u:
                take[u] += skip[v]
                skip[u] += max(take[v], skip[v])
    print(max(take[0], skip[0]))

if __name__ == "__main__":
    main()
```

2.9 Common wrong answers

Taking a node and also allowing chosen children.
Processing nodes in BFS order instead of postorder.
Using int for large happiness sums.

3 B. Road Pairing

3.1 Restatement

Find the largest set of edges in a tree such that no two chosen edges share a vertex.

3.2 Observations

For a node u , first assume it is not matched to a child. Then every child can use its best state. To match u to one child v , replace v 's best state by $\text{free}[v]$ and add one chosen edge.

3.3 Algorithm

```
base = sum(max(free[v], used[v])) over children
free[u] = base
used[u] = max(base, base - max(free[v], used[v]) + free[v] + 1)
```

3.4 Correctness proof

Theorem. The transition computes the maximum matching in every subtree.

Proof. Induct on subtree height. If u is not matched to a child, children are independent and contribute their best values. If u is matched to a child v , no other child edge incident to u can be chosen, and v cannot be matched inside its child subtrees; hence $\text{free}[v] + 1$ replaces v 's previous best contribution. Trying every child covers every possible edge incident to u . $\square$

3.5 Complexity

```
Time:  $O(n)$ 
Memory:  $O(n)$ 
```

3.6 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<vector<int>> adj(n);
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        cin >> u >> v;
        --u; --v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> parent(n, -1), order;
    parent[0] = 0;
    order.push_back(0);
    for (int i = 0; i < (int)order.size(); i++) {
        int u = order[i];
        for (int v : adj[u]) if (parent[v] == -1) {
            parent[v] = u;
            order.push_back(v);
        }
    }
    vector<int> free(n, 0), used(n, 0);
    for (int idx = n - 1; idx >= 0; idx--) {
        int u = order[idx];
        int base = 0;
        vector<int> children;
        for (int v : adj[u]) if (parent[v] == u) {
            children.push_back(v);
            base += max(free[v], used[v]);
        }
        free[u] = base;
        int best = base;
        for (int v : children) {
            best = max(best, base - max(free[v], used[v]) + free[v] + 1);
        }
        used[u] = best;
    }
    cout << max(free[0], used[0]) << '\n';
    return 0;
}

```

3.7 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    adj = [[] for _ in range(n)]
    for _ in range(n - 1):
        u = next(it) - 1
        v = next(it) - 1
        adj[u].append(v)
        adj[v].append(u)
    parent = [-1] * n
    parent[0] = 0
    order = [0]
    for u in order:
        for v in adj[u]:
            if parent[v] == -1:
                parent[v] = u
                order.append(v)
    free = [0] * n
    used = [0] * n
    for u in reversed(order):
        base = 0
        children = []
        for v in adj[u]:
            if parent[v] == u:
                children.append(v)
                base += max(free[v], used[v])
        free[u] = base
        best = base
        for v in children:
            cand = base - max(free[v], used[v]) + free[v] + 1
            if cand > best:
                best = cand
        used[u] = best
    print(max(free[0], used[0]))

if __name__ == "__main__":
    main()
```

4 C. Tree Colorings

4.1 Restatement

Color every vertex white or black so that no edge has two black endpoints. Count valid colorings modulo 1_000_000_007.

4.2 Observations

If u is white, each child may be white or black. If u is black, every child must be white.

4.3 Algorithm

```
white[u] = product(white[v] + black[v])
black[u] = product(white[v])
answer = white[root] + black[root]
```

4.4 Correctness proof

Theorem. The DP counts exactly all valid colorings of each rooted subtree.

Proof. Induct on subtree height. A leaf has one white and one black coloring. For an internal node, fixing the color of u makes child subtrees independent. White u imposes no black constraint on children, while black u forces every child root white. Multiplying counts independent choices across children. $\square$

4.5 Complexity

```
Time:  $O(n)$ 
Memory:  $O(n)$ 
```

4.6 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    const long long MOD = 1000000007LL;
    int n;
    if (!(cin >> n)) return 0;
    vector<vector<int>> adj(n);
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        cin >> u >> v;
        --u; --v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> parent(n, -1), order;
    parent[0] = 0;
    order.push_back(0);
    for (int i = 0; i < (int)order.size(); i++) {
        int u = order[i];
        for (int v : adj[u]) if (parent[v] == -1) {
            parent[v] = u;
            order.push_back(v);
        }
    }
    vector<long long> white(n, 1), black(n, 1);
    for (int idx = n - 1; idx >= 0; idx--) {
        int u = order[idx];
        for (int v : adj[u]) if (parent[v] == u) {
            white[u] = white[u] * ((white[v] + black[v]) % MOD) % MOD;
            black[u] = black[u] * white[v] % MOD;
        }
    }
    cout << (white[0] + black[0]) % MOD << '\n';
    return 0;
}

```

4.7 Full Python solution

```

import sys

MOD = 1_000_000_007

```

```

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    adj = [[] for _ in range(n)]
    for _ in range(n - 1):
        u = next(it) - 1
        v = next(it) - 1
        adj[u].append(v)
        adj[v].append(u)
    parent = [-1] * n
    parent[0] = 0
    order = [0]
    for u in order:
        for v in adj[u]:
            if parent[v] == -1:
                parent[v] = u
                order.append(v)
    white = [1] * n
    black = [1] * n
    for u in reversed(order):
        for v in adj[u]:
            if parent[v] == u:
                white[u] = white[u] * (white[v] + black[v]) % MOD
                black[u] = black[u] * white[v] % MOD
    print((white[0] + black[0]) % MOD)

if __name__ == "__main__":
    main()

```

5 Testing and carry-forward

The section checker runs:

```
CP_TARGET=solution python3 sections/26_tree_dp_basics/check.py
```

Tests include single-node trees, paths, stars, and random trees.

Carry forward:

```
Root the tree.  
Define states around the node's relationship to its parent/children.  
Process children before parents.  
Combine child subtrees independently.
```